

Specyfikacja Projektu Zaliczenowego (Programowanie w Języku Java)

[Twoje Imię i Nazwisko]

[Data]

1. Tytuł Projektu

Przedstaw nazwę projektu, która powinna być jednoznaczna, krótka i oddawać główną ideę aplikacji.

[Podaj nazwę projektu]

2. Opis i Cele Projektu

Określ, jaki problem rozwiązuje projekt, do czego służy oraz jakie są jego główne cele funkcjonalne.

Przykład: Projekt ma na celu stworzenie aplikacji [opis projektu], która umożliwia użytkownikom [co umożliwia]. Główne cele projektu:

- Bezpieczna rejestracja i logowanie użytkowników.
- Przypisywanie zadań, etykiet, priorytetów.
- Współpraca w czasie rzeczywistym (WebSocket).
- Statystyki wykonania zadań.

3. Wymagania Funkcjonalne

Lista konkretnych funkcji, które aplikacja powinna oferować użytkownikowi końcowemu.

Przykład:

- Rejestracja, logowanie, zarządzanie sesją.
- Obsługa zadań: tworzenie, edycja, usuwanie, filtrowanie.
- Przypisywanie użytkowników, etykiet i terminów.
- Role użytkowników (admin, zwykły użytkownik).
- Wyszukiwanie i filtrowanie danych.
- Aktualizacje w czasie rzeczywistym.

4. Wymagania Niefunkcjonalne

Wymagania techniczne, jakościowe i pozafunkcjonalne, które aplikacja powinna spełniać.

Przykład:

- **Wydażność:** Czas przetwarzania danych poniżej ... sekundy.
- **Skalowalność:** Obsługa minimum ... użytkowników równocześnie.
- **Bezpieczeństwo:** HTTPS, hasła szyfrowane (bcrypt).
- **Użyteczność:** Interfejs zgodny z WCAG (AA).
- **Zgodność:** Responsywny interfejs (desktop, tablet, mobilne).

5. Architektura Systemu

Opis architektury systemu, jego głównych komponentów oraz zależności między nimi.

Przykład: Typ architektury: **Klient-Serwer (MVC)**.

Schemat:

- Frontend: React.js + TailwindCSS
- Backend: Java (framework zależny od implementacji)
- Baza danych: PostgreSQL lub MongoDB
- WebSocket: np. STOMP/WebSocket
- Autoryzacja: JWT / OAuth2

6. Zastosowane Algorytmy

Opis algorytmów i struktur danych wykorzystanych w aplikacji do realizacji funkcjonalności.

Przykład:

- **Algorytm sortowania zadań:** Sortowanie zadań po terminie, priorytecie lub statusie — wykorzystanie np. sortowania przez porównanie z wykorzystaniem Comparator w Javie.
- **Algorytmy uwierzytelniania:** Tokenizacja i walidacja JWT (algorytm HMAC-SHA256), szyfrowanie haseł z użyciem bcrypt.
- **Filtrowanie i wyszukiwanie:** Przeszukiwanie listy zadań według etykiet, tekstu lub użytkownika — implementacja z wykorzystaniem filtrów i indeksowania.

- **Zarządzanie sesją:** Wygasanie tokenów na podstawie czasu (timestamp + TTL), middleware do odświeżania sesji.
- **Algorytmy notyfikacji:** Sprawdzanie zbliżających się terminów i wyzwalanie powiadomień.
- **Algorytmy kolejowania wiadomości:** W WebSocket można zastosować buforowanie i kolejowanie aktualizacji do wielu klientów jednocześnie.

7. Struktura Klas

Opisuje strukturę klas, ich odpowiedzialności oraz podział logiczny na warstwy w aplikacji.

Przykład:

Model

- **User** – identyfikator, imię, email, hasło, rola.
- **Task** – id, tytuł, opis, termin, status, przypisany użytkownik.
- **Tag** – id, nazwa, kolor, powiązanie z zadaniami.

Serwisy

- **UserService** – rejestracja, logowanie, zarządzanie rolami.
- **TaskService** – logika zarządzania zadaniami.
- **NotificationService** – generowanie powiadomień.
- **AuthService** – JWT, walidacja, bezpieczeństwo.

Kontrolery

- **UserController** – REST API użytkownika.
- **TaskController** – REST API zadań.
- **AuthController** – logowanie i uwierzytelnianie.

Dodatkowe komponenty

- **WebSocketHandler** – aktualizacje w czasie rzeczywistym.
- **DTO** – obiekty wymiany danych (Data Transfer Object).
- **Repository** – interfejsy do bazy danych (np. JPA/Hibernate).

8. Wykorzystane Technologie i Biblioteki

Opis technologii oraz narzędzi użytych w projekcie.

Przykład:

Frontend:

React, React Query

Backend:

Java, WebSocket

Baza Danych:

PostgreSQL lub MongoDB

DevOps:

Docker, GitHub Actions

9. API i Modele Danych

Przykładowe punkty końcowe API oraz struktura danych wymienianych między systemami.

Przykład:

Przykładowe Endpointy:

```
GET /api/tasks
POST /api/tasks
PUT /api/tasks/:id
DELETE /api/tasks/:id
```

Model Danych:

```
{
  "id": "uuid",
  "tytul": "string",
  "opis": "string",
  "termin": "timestamp",
  "priorytet": "enum(niski, sredni, wysoki)",
  "etykiety": ["string"],
  "zrealizowane": "boolean",
  "przypisanyDo": "user_id"
}
```

10. Interfejs Użytkownika

Opis interfejsu graficznego aplikacji, jego struktury, logiki działania oraz dostępnych ekranów.

Przykład:

Główne widoki:

- **Ekran logowania i rejestracji:** Formularze z walidacją danych, możliwość odzyskania hasła.
- **Pulpit użytkownika:** Przegląd aktualnych zadań, kalendarz, szybki dostęp do przypisanych zadań.
- **Zarządzanie zadaniami:** Widok listy zadań z możliwością sortowania, filtrowania, edytowania i usuwania.
- **Tworzenie i edycja zadań:** Formularz z polami na tytuł, opis, etykiety, termin, przypisanie użytkownika.
- **Powiadomienia:** Wyskakujące alerty o nadchodzących terminach i zmianach w zadaniach (real-time).
- **Panel administracyjny:** Zarządzanie użytkownikami, rolami, dostępem.

Elementy UX/UI:

- Responsywność — dostosowanie do urządzeń mobilnych.
- Konsekwentna kolorystyka, nawigacja boczna lub górna.
- Łatwość obsługi — maksymalnie 3 kliknięcia do najważniejszych funkcji.
- Tryb ciemny/jasny — dostępny z poziomu ustawień.

Technologie UI:

React, TailwindCSS, React Router, Headless UI, ikony Lucide.

11. Etapy Realizacji / Harmonogram

Plan wdrożenia projektu z podziałem na fazy i szacowanym czasem realizacji.

Przykład:

Faza	Opis	Czas
Planowanie	Określenie wymagań, wybór technologii	1 tydzień
Backend	Budowa API, modele danych	2 tygodnie
Frontend	Interfejs, integracja z API	2 tygodnie

Testy i realtime	WebSocket, testy jednostkowe i integracyjne	1 tydzień
Wdrożenie	Konfiguracja Docker, CI/CD, serwer	1 tydzień

12. Testowanie

Sposoby testowania aplikacji, typy testów oraz używane narzędzia.

Przykład:

- Testy jednostkowe (logika backendu i frontend).
- Testy integracyjne (API).
- Testy E2E – np. Cypress, Selenium.
- Testy obciążeniowe – np. JMeter, k6.

13. Ryzyka i Środki Zaradcze

Identyfikacja potencjalnych zagrożeń projektowych i strategii ich ograniczenia.

Przykład:

Ryzyko	Wpływ	Zarządzanie
Nadmierna liczba funkcji	Wysoki	Zablokowanie wymagań po planowaniu
Błędy bezpieczeństwa	Wysoki	Audyty kodu, sprawdzone biblioteki
Opóźnienia	Średni	Praca iteracyjna (Agile)

14. Kryteria Sukcesu

Opisz po czym poznać, że projekt zakończył się sukcesem.

Przykład:

- Wszystkie funkcje działają zgodnie ze specyfikacją.
- Pokrycie testami > 80%.
- Brak błędów krytycznych na produkcji.
- Pozytywna opinia użytkowników (średnia ocena >4.5).

15. Szacunkowe Nakłady Pracy (Workload Estimation)

Osobne oszacowanie czasu pracy dla poszczególnych modułów i funkcji.

Przykład: Całkowity szacowany czas pracy: **160 godzin roboczych**.

Moduł / Zadanie	Opis	Szac. czas (h)
Analiza i projektowanie	Wymagania, mockupy, wybór technologii	12
Backend API (CRUD, JWT)	Modele danych, logika biznesowa	35
Frontend UI (React)	Komponenty, interakcje z API	30
Uwierzytelnianie i sesje	Login, rejestracja, autoryzacja	15
WebSocket – realtime	Notyfikacje i live updates	10
Testy (unit + E2E)	Pokrycie testami kluczowych ścieżek	15
Docker / CI / Deployment	Konfiguracja środowiska, produkcja	10
Dokumentacja techniczna	README, API schema, komentarze	8
Bufor czasowy / błędy	Debugowanie, nieprzewidziane problemy	25